Preprocessor

Preprocessor: program that processes source code before compilation

File Inclusion

```
#include "filename" // use when the file exists locally
#include <filename> // use for the standard C libraries
```

Macro Substitution

```
#define name replacement_text
```

`name` will be replaced by `replacement_text` like how you assign values to variable

A long definition can be written by using a `\` at the end of the line

Substitution doesn't work for quoted string

```
#define forever for (;;) // infinite loop
#define max(a, b) ((a) > (b) ? (a) : (b))
/* looks like function invocation, but a and b will be replaced by values,
e.g., x = max((p + q), (r + s)); will be replaced by x = ((p + q) > (r + s)
? (p + q) : (r + s)); */
```

This macro will work fine as long as the arguments are treated consistently. However, the problem is that the expression evaluate twice; it will gives wrong values if we use increment.

```
x = max(i++, j++);
```

In this case, it will be incremented twice

Consider the macro

```
#define square(x) x * x
```

When we pass `z + 1` as the parameter, the macro would return `z + 1 * z + 1 = z + z + 1 = 2 * z + 1` rather than `(z + 1) * (z + 1)`

So, to get the intended behavior, we must use extra parentheses

```
#define square(x) (x) * (x)
```

`#undef` tells the processsor to undefine the macro

```
#undef getchar
```

Formal parameters are not replaced with quoted string. However, if `#` is present in the replacement text, then the combination will be exxpanded into quoted string

Consider

```
#define dprint(exp) printf(#exp " = &g\n", exp)
```

The invocation of `dprint(x/y)` will be

```
printf("x/y = &g\n", x/y);
```

Conditional Inclusion

We can use conditional statement in preprocessor with `#if`

Consider we have a `hdr.h` file

If we want to make sure the file is included, we can use

```
#if !define(HDR)
#define HDR
// contents
#endif
```